

Benjamin Garcia

(626) 246-4778 | bentgarcia05@gmail.com | [GitHub](#) | [LinkedIn](#) | [Portfolio](#)

EDUCATION

University of California, Los Angeles
Bachelor of Science, Computer Science

Los Angeles, CA
Expected Graduation: Dec. 2027

TECHNICAL SKILLS

Languages: TypeScript, Python, Java, C, C++, SQL, JavaScript, HTML/CSS

Frameworks & Libraries: Next.js, React, Tailwind CSS, FastAPI, Prisma, Node.js, Zod, Zustand, TanStack Query, React Flow

Databases & Infrastructure: PostgreSQL, Supabase, Firestore, Redis, Docker, Git, Linux/Unix, AWS Lambda, Kubernetes

EXPERIENCE

Undergraduate Researcher

Dec. 2025 – Present

UCLA Human-Computer Interaction Research Lab

Los Angeles, CA

- Building **HI-COS**, an interactive system for scientific hypothesis generation, to enable **cross-domain literature search**, **hypothesis comparison**, and **evidence-trail inspection** by developing the interface in **React, TypeScript, and Next.js**.
- Increasing transparency of AI-recommended protein interaction pathways by surfacing **interaction graphs**, **path-level reasoning**, retrieved **STRING annotations**, and **edge-level explanations** through human-in-the-loop workflows over a knowledge-graph-based retrieval pipeline.
- Advancing AI-assisted scientific discovery workflows by exploring **multi-agent LLM systems** in **Python** to support iterative hypothesis generation, structured evaluation, and interpretable explanation rendering.

Software Engineer Intern

Mar. 2025 – Sep. 2025

Todd Agriscience

Remote, Part-time

- Delivered **AI-driven crop insight dashboards** for **5–10** clients by architecting full-stack systems in Next.js, Tailwind CSS, and Framer Motion, integrated with **Palantir Foundry** and **AWS Lambda**.
- Developed dynamic data pipelines visualizing **20+** agricultural data streams including soil, irrigation, weather, and yield data to enable real-time farm management decisions.

Research and Development Engineer Intern

Jul. 2025 – Aug. 2025

Bonterra

Remote

- Prototyped an **agentic AI system** for nonprofit event analytics using tool-calling frameworks and **Model Context Protocols (MCPs)**, modeling attendance, retention, and fundraising across datasets; benchmarked **5+** agent frameworks.
- Developed and tested a **multi-tool orchestration layer** enabling autonomous agent collaboration, improved retrieval latency by **22%**, and validated tool-selection policies for scaling across nonprofit datasets.

Software Engineer Intern

May 2025 – Jun. 2025

TensorStar

Remote

- Engineered **real-time agentic UIs** with Next.js, Redux, and WebSockets, supporting **100+** concurrent users with **sub-100ms** latency while collaborating with a **10–15-engineer team** across frontend, ML, and infrastructure.
- Built secure credential-submission workflows integrated with **HashiCorp Vault** and a **Python + Redis backend**, unifying authentication across **50+** enterprise data sources including AWS Glue, Snowflake, MongoDB, Postgres, Airflow, dbt, and Spark.

PROJECTS

GitProof | *Next.js, TypeScript, PostgreSQL, Prisma, Gemini API, GitHub GraphQL API*

Dec. 2025 – Present

- Architected a full-stack developer portfolio platform transforming GitHub activity into recruiter-ready insights, processing **100+** repositories per user through a type-safe pipeline integrating the GitHub GraphQL API, PostgreSQL, and Prisma ORM.
- Designed a **3-factor** weighted impact scoring algorithm (**0–50** scale) combining logarithmic popularity scaling, recency decay across **5** time tiers, and project maturity signals to rank diverse portfolios with user-level weighted aggregation.
- Built an **AI-powered insights and content system** using Google Gemini 2.5 Flash to generate project descriptions, professional bios, and personalized portfolio feedback from repository metadata, documentation quality, and contribution patterns.

Logit | *Next.js, TypeScript, PostgreSQL, Prisma, React, Recharts, Anthropic API*

Mar. 2026 – Present

- Built a full-stack workout journal with authenticated logging, split planning, AI-assisted split generation, dashboard analytics, exercise history, and public profiles across a normalized **11-model** PostgreSQL schema.
- Engineered transaction-safe workout and split mutations across **12** API route files, combining payload validation, pound-based weight normalization, computed totals, derived read models, user-scoped cache invalidation, and trusted-origin protections.
- Added **23** TypeScript test files covering **76** cases across workout math, date-only scheduling, split rules, AI draft validation, payload normalization, read-model sync, service behavior, and export formatting.